Lecture 41: Deep Learning (Part-6)

Minor Project: Sentiment Analysis on IMDB Reviews with PyTorch RNN

Welcome to the comprehensive, publication-grade documentation for **Lecture 41: Deep Learning (Part-6)**. This curriculum covers an end-to-end practical minor project in PyTorch: building a Recurrent Neural Network (RNN) from scratch for binary sentiment classification on the benchmark **IMDB Large Movie Review Dataset** ($50,000$ reviews), complete with text normalization, regular expressions, linguistic tokenization, stopwords removal, Porter Stemming, TF-IDF vectorization, PyTorch DataLoaders, BCELoss optimization, live real-world inference, and a comparative study of CNN vs RNN paradigms on sequential vision tasks (MNIST).

---

1. Master Curriculum Architecture

```
lecture_41/
├── Lecture_Code/
│   ├── Codes.ipynb                        <- Complete end-to-end executable PyTorch project
│   └── Resources/
│       ├── 00) Contents TEAMWORK.jpeg     <- Syllabus overview
│       ├── 9 Handwritten Digit Classification using CNN & RNN.png <- CNN vs RNN reference
│       └── 10 MNIST Dataset in PyTorch.png <- MNIST vision sequential reference
│
├── Notes_41.01/  <- Minor Project: Sentiment Analysis on IMDB Reviews with PyTorch RNN
├── Notes_41.02/  <- Text Normalization & Regex Preprocessing (Case Folding, URLs, Punctuations & HTML)
├── Notes_41.03/  <- Linguistic Tokenization & Stopwords Removal with NLTK
├── Notes_41.04/  <- Morphological Stemming vs Lemmatization & Vocabulary Construction
├── Notes_41.05/  <- Text Vectorization using TF-IDF (Term Frequency - Inverse Document Frequency)
├── Notes_41.06/  <- PyTorch Data Pipelines: TensorDataset, DataLoader, Batching & Splitting
├── Notes_41.07/  <- Designing the Recurrent Neural Network (RNN) Architecture in PyTorch
├── Notes_41.08/  <- Optimization, Loss Function & The Recurrent Training Loop
├── Notes_41.09/  <- Evaluation, Metrics & Real-World Inference on Unseen Reviews
├── Notes_41.10/  <- Comparative Study: Sequence Processing with RNNs vs Spatial Convolutions with CNNs
│
└── Practice_Problems/
    ├── README.md                          <- 15 High-Yield Technical Problems (English)
    ├── README_HINGLISH.md                 <- 15 High-Yield Technical Problems (Hinglish)
    ├── Solutions.md                       <- Full Mathematical & Code Solutions (English)
    └── Solutions_HINGLISH.md              <- Full Mathematical & Code Solutions (Hinglish)
```

---

2. Overview of Modules

Submodule	Core Theoretical & Practical Concepts	Key Code / Formulas	Visual Artifact
Notes_41.01	Minor Project Overview, IMDB dataset (50K reviews), Deduplication ($50\text{K} \rightarrow 49,582$), EDA	`df.drop_duplicates(inplace=True)`	`sentiment_analysis_imdb_pipeline.png`
Notes_41.02	Text Normalization, Regex module `re`, URL stripping, HTML non-greedy stripping, Caret punctuation removal	`re.sub(r'http\S+', '', text)` `re.sub(r'[^A-Za-z0-9\s]', '', text)`	`regex_text_preprocessing_pipeline.png`
Notes_41.03	Linguistic Tokenization, NLTK punkt, Stopwords corpus filtering, Zipf's Law ($f(w) \propto 1/r$)	`word_tokenize(text)` `stopwords.words('english')`	`nltk_tokenization_stopwords_workflow.png`

Submodule	Core Theoretical & Practical Concepts	Key Code / Formulas	Visual Artifact
Notes_41.04	Morphological Normalization, Porter Stemmer cascading rules, Over/under-stemming error modes	`ps = PorterStemmer()` `ps.stem(token)`	`porter_stemming_vs_lemmatization.png`
Notes_41.05	Text Vectorization, TF-IDF statistical weighting, `max_features=5000` memory constraint, CSR matrix	$\text{TF} \times \text{IDF}$, Shape: $(49582, 5000)$ `X.toarray()`	`tfidf_vectorization_space_matrix.png`
Notes_41.06	PyTorch Pipelines, Train/Test Split (80/20), FloatTensors, `TensorDataset`, `DataLoader(batch=64, shuffle=True)`	620 batches per epoch	`pytorch_tensordataset_dataloader_pipeline.png`
Notes_41.07	Custom `nn.Module` RNN, Hidden state dynamics, 3D tensor unsqueeze, Linear readout, $656,641$ parameters	`nn.RNN(5000, 128, batch_first=True)` `out[:, -1, :]`	`pytorch_rnn_sentiment_architecture.png`
Notes_41.08	Binary Cross-Entropy Loss (`BCELoss`), Sigmoid calibration, Adam optimizer, 10-Epoch loop	$\mathcal{L} = -[y \log(p) + (1-y) \log(1-p)]$ `loss.backward()`	`rnn_training_optimization_workflow.png`
Notes_41.09	Evaluation protocol, `model.eval()`, `torch.no_grad()`, threshold $\tau = 0.5$, test accuracy $\approx 86\% - 88\%$, Live inference	`predict_sentiment(text)`	`model_evaluation_and_inference_pipeline.png`
Notes_41.10	Comparative Study: 2D Spatial Convolutions (CNN) vs 1D Sequential Processing (RNN) on MNIST (28×28)	Spatial 2D Grid vs 28 Temporal Row Steps	`cnn_spatial_vs_rnn_sequential_mnist.png`

---

3. Educational Handwritten Visual System

Every single concept across this curriculum is accompanied by a **notebook-friendly, hand-drawn educational diagram** created in standard blue/black pen notebook styling (`Comic Sans MS`) with clean annotations, mathematical labels, and step-by-step unrolled flows. All instructor webcam and video keyframe images have been strictly omitted.